

FAKE NEWS DETECTION SYSTEM

Complete Project Report

Technology: React.js (Vite) + Django REST Framework + Python Machine Learning

ML Approach: TF-IDF + PassiveAggressiveClassifier

Model Accuracy Achieved: 99.47% on the training/test evaluation reported during development

Final Project Documentation

Table of Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Proposed Solution
- 6. Technology Stack
- 7. System Architecture
- 8. Machine Learning Methodology
- 9. Dataset and Training
- 10. Backend Implementation
- 11. Authentication and Authorization
- 12. Database and Prediction History
- 13. Frontend Implementation
- 14. User Dashboard
- 15. Admin Dashboard
- 16. API Endpoints
- 17. Project Workflow
- 18. Testing
- 19. Error Handling and Debugging
- 20. Security Considerations
- 21. Project Structure
- 22. Limitations
- 23. Future Enhancements
- 24. Conclusion

1. Abstract

This project presents a web-based Fake News Detection System designed to classify submitted news content as **Real** or **Fake** using a machine-learning model.

The application combines a React.js frontend with a Django backend. Users can submit news articles, receive a prediction and confidence-related score, review previous predictions, search and filter history, delete individual records, clear history, and export prediction records to PDF or CSV.

The backend contains the machine-learning pipeline, REST APIs, authentication, role-based administration, prediction history storage, and user-management functionality. JWT-based authentication is used for protected operations.

The trained model achieved an accuracy of **99.47%** during the reported model evaluation. The application also includes an administrator area for viewing and managing registered users.

2. Introduction

False and misleading information can spread rapidly through digital platforms. Manual verification of every article is time-consuming, which motivates the use of automated text-classification systems.

The project applies natural-language text processing and supervised machine learning to transform news content into numerical TF-IDF features and classify it with a PassiveAggressiveClassifier.

The final application provides a complete workflow rather than only a machine-learning script: authentication, prediction, persistence, analytics, history management, user management, and export functionality are integrated into one web application.

3. Problem Statement

Users need a convenient way to obtain an automated indication of whether a piece of news resembles real or fake news according to a trained classification model.

The system must accept textual news content, preprocess it consistently with training data, generate a prediction through the trained model, return the result through an API, and preserve the result for later analysis.

4. Objectives

Build a machine-learning based fake-news classification model.

Create a REST API for submitting news and receiving predictions.

Develop a responsive React dashboard for users.

Store prediction history in the backend database.

Provide statistics such as total predictions, real-news count, fake-news count, and a dashboard accuracy-style metric.

Provide visual analytics through a distribution chart and prediction trend chart.

Implement JWT authentication with registration and login.

Provide protected profile and password-management operations.

Implement staff/admin authorization and an admin user-management dashboard.

Allow users to search, filter, delete, clear, and export prediction history.

5. Proposed Solution

The solution is divided into four logical layers: the React presentation layer, Django REST API layer, machine-learning layer, and database layer.

The React application communicates with Django using HTTP requests through a centralized API client. Django validates requests, invokes the trained model for prediction, stores prediction history, and returns JSON responses. Protected endpoints require authentication, while admin-only endpoints additionally verify staff privileges.

6. Technology Stack

Frontend: React.js, Vite, CSS, React Icons, Chart.js/react-chartjs-2, Recharts, React Toastify.

Backend: Python, Django, Django REST Framework, Simple JWT.

Machine Learning: pandas, scikit-learn, TF-IDF Vectorizer, PassiveAggressiveClassifier, joblib.

Database: Django-supported relational database used for application records and prediction history.

Development: Visual Studio Code, Postman, Python virtual environment, Windows development environment.

7. System Architecture

The user interacts with the React frontend. The frontend sends requests to Django REST endpoints. Django handles authentication, authorization, prediction requests, history operations, and administrative operations. The prediction endpoint sends cleaned text through the saved TF-IDF vectorizer and machine-learning model. Prediction results are persisted through the PredictionHistory model.

High-level flow: User → React UI → Django REST API → preprocessing → TF-IDF vectorization → ML model → prediction/score → database → JSON response → React dashboard.

8. Machine Learning Methodology

The model uses supervised text classification. The dataset contains separate fake and true news CSV files. During training, fake records are assigned label 0 and true records are assigned label 1.

The title and article text are combined into a content field. Text is cleaned using the project's preprocessing utility. TF-IDF converts the cleaned text into numerical features. The classifier learns the relationship between these features and the target labels.

During prediction, the submitted article follows the same cleaning and vectorization process. The classifier then produces a class prediction and a decision score. The application maps class 1 to **Real** and the other class to **Fake**.

9. Dataset and Training

The training script loads **Fake.csv** and **True.csv**, labels them, concatenates the records, and shuffles the combined data.

The project initially explored NLTK-based preprocessing but moved to a scikit-learn-based preprocessing pipeline because of an environment/import issue involving the regex dependency. The final implementation therefore uses the project's cleaning utility together with scikit-learn TF-IDF processing.

The reported training/evaluation output was **99.47% model accuracy**. The trained artifacts were saved as **fake_news_model.pkl** and **tfidf_vectorizer.pkl** under the backend model directory.

10. Backend Implementation

The Django backend exposes REST endpoints for authentication, prediction, prediction history, profile management, password changes, and administrative user management.

The prediction API accepts a POST request containing a **news** field. It calls the prediction function, obtains the prediction and score, stores the result in PredictionHistory, and returns the result as JSON.

The history API returns stored prediction records ordered by newest creation time. Individual history records can be deleted and the entire history can be cleared.

11. Authentication and Authorization

Registration uses Django's built-in User model and **create_user()**, ensuring that submitted passwords are stored using Django's password hashing mechanism.

Login verifies the supplied username and password using Django authentication. Successful login returns an access token and refresh token generated by Simple JWT, along with basic user and role information.

Protected operations use **IsAuthenticated**. Administrative endpoints use the custom **IsAdminUserRole** permission, which requires an authenticated user whose **is_staff** value is true.

The project also provides a protected change-password endpoint. It checks the current password, prevents reuse of the same password, validates the new password with Django's password validators, and saves it securely.

12. Database and Prediction History

Prediction results are stored using a PredictionHistory model containing the submitted news, prediction, score, and creation timestamp, along with its database identifier.

The React dashboard fetches this history and derives total predictions, real count, fake count, and the displayed accuracy-style percentage from the returned records.

History management includes search by news content, filtering by Real/Fake, individual deletion, complete history clearing, and export of the currently filtered records.

13. Frontend Implementation

The frontend is built using React components and Vite. The main Home page combines navigation, hero content, prediction input, prediction results, statistics, analytics, and history.

The application uses reusable components such as Navbar, Hero, NewsForm, PredictionCard, History, StatsCard, PieChart, and PredictionLineChart.

Toast notifications provide feedback for successful predictions, validation warnings, history deletion, history clearing, and export operations. A loading state and loader component are displayed while prediction requests are being processed.

14. User Dashboard

The dashboard contains a Prediction Overview section with Total Predictions, Real News, Fake News, and Accuracy cards.

The Check News section allows the user to paste an article and submit it for detection. The interface displays an active detection status and the prediction result after the backend responds.

The Prediction Analytics section contains a prediction distribution pie chart and a prediction trend line chart. The history section allows users to review and manage previous analyses.

The prediction card displays the classification, an icon, a confidence-style progress indicator derived from the returned score, analysis status, and date.

15. Admin Dashboard

The application includes an administrator area for managing registered users.

The admin users page retrieves users through the protected admin users API, sorts them by ID, and supports searching by name, username, or email.

Each user record displays ID, name, username, email, and joined date. The interface also provides a delete action with confirmation before removal.

The backend permission class restricts the users endpoint to authenticated staff users.

16. API Endpoints

POST /api/auth/register/ Registers a new user and returns JWT tokens plus basic user information.

POST /api/auth/login/ Authenticates a user and returns access/refresh JWT tokens.

GET/PUT /api/auth/profile/ Retrieves or updates the authenticated user's profile.

POST /api/auth/change-password/ Changes the authenticated user's password.

GET /api/auth/users/ Returns registered users for authorized staff users.

GET/PUT/DELETE /api/auth/users/<pk>/ Retrieves, updates, or deletes a selected user for authorized staff users.

POST /api/predict/ Classifies submitted news and stores the prediction.

GET /api/history/ Returns prediction history.

DELETE /api/history/<id>/ Deletes one prediction record.

DELETE /api/clear-history/ Clears prediction history.

17. Project Workflow

User registration: User submits registration details → Django validates data → User is created → JWT tokens are generated.

User login: Credentials are submitted → Django authenticates the user → JWT access/refresh tokens are returned → frontend uses authentication for protected APIs.

News prediction: User enters news → frontend sends POST request → Django validates input → preprocessing → TF-IDF vectorization → model prediction → result saved → JSON response → prediction card and dashboard history update.

History management: Frontend loads history → user searches/filters → selected records can be deleted or all records can be cleared → dashboard state refreshes.

Administration: Staff user accesses admin users API → permission class verifies staff role → users are displayed → administrator can search and delete users.

18. Testing

Backend authentication and prediction APIs were tested using API requests and Postman during development.

The login flow was debugged using server-side diagnostic output. A case where the user existed and was active but the supplied password did not match was identified through the **check_password()** result.

The Django server also exposed an import error when **AdminUserSerializer** was referenced without being defined in `accounts.serializers`. This was corrected by aligning the serializer imports with the available serializers.

A later startup error reported **IsAdminUserRole** as undefined. The custom permission class was added/imported so protected administrative views could load correctly.

The frontend was tested for successful prediction, history refresh, charts, search/filter behavior, deletion, clear-history functionality, and user dashboard behavior.

19. Error Handling and Debugging

The frontend checks whether an error has an HTTP response and displays an appropriate toast message for server-side errors versus connection failures.

The backend returns appropriate HTTP status codes for invalid input, authentication failures, successful creation, successful retrieval, and successful updates/deletions.

The machine-learning training process produced a FutureWarning because PassiveAggressiveClassifier is deprecated in a future scikit-learn release. This warning does not itself prevent the current model from running, but the classifier can be migrated later to the recommended SGDClassifier configuration.

20. Security Considerations

Passwords are created and changed using Django's secure password hashing methods rather than storing plaintext passwords.

JWT tokens are used for stateless API authentication.

Protected endpoints require authentication, while administrative endpoints require staff authorization.

Password values should never be printed in production logs. The development login debugging was designed to print only whether a password was provided and whether Django's password check succeeded.

Production deployment should additionally use HTTPS, secure token storage, environment variables for secrets, restricted CORS settings, proper database credentials, and production-grade logging.

21. Project Structure

backend/ contains the Django project and applications.

backend/accounts/ contains authentication, profile, user serializers, URLs, views, and permissions.

backend/dashboard/ contains dashboard-related backend endpoints and administrative functionality.

backend/detection/ contains machine-learning training and prediction utilities.

backend/model/ contains the trained model and TF-IDF vectorizer artifacts.

frontend/src/ contains React pages, components, services, utilities, and styles.

frontend/src/components/Charts/ contains the prediction analytics charts.

frontend/src/components/History/ contains prediction-history UI and styling.

frontend/src/components/NewsForm/ contains the news submission and prediction flow.

frontend/src/components/PredictionCard/ displays prediction results.

frontend/src/components/Admin/ contains administrator UI such as the users management page and sidebar.

22. Limitations

The displayed accuracy-style dashboard metric is calculated from the prediction history as the proportion of records classified as Real. It should not be interpreted as the machine-learning model's scientific accuracy on unseen data.

The reported 99.47% accuracy depends on the dataset and evaluation procedure used during development. It should not be treated as universal real-world accuracy.

The model is text-based and may struggle with new topics, satire, ambiguous writing, intentionally manipulated text, or news outside the distribution of the training dataset.

The current classifier uses `PassiveAggressiveClassifier`, which is deprecated in future scikit-learn versions and should eventually be migrated.

23. Future Enhancements

Migrate the deprecated `PassiveAggressiveClassifier` to the recommended `SGDClassifier` configuration.

Add a formal train/validation/test split and report precision, recall, F1-score, confusion matrix, and ROC-AUC where appropriate.

Introduce model versioning and periodic retraining with newer verified datasets.

Improve explainability by showing important words/features influencing classification.

Add pagination and server-side filtering for large prediction histories.

Add richer admin analytics, user status management, and audit logs.

Deploy the frontend and backend using production infrastructure with HTTPS and secure environment configuration.

Improve mobile responsiveness and accessibility.

24. Conclusion

The Fake News Detection System successfully integrates machine learning with a full-stack web application. The project demonstrates how a text-classification model can be exposed through REST APIs and consumed by a modern React interface.

The completed system provides user registration and login, protected APIs, news prediction, prediction history, dashboard statistics, charts, export functions, password management, and staff-based administration.

With the reported model evaluation of 99.47% and a functional end-to-end application workflow, the project provides a strong foundation for further improvements in model evaluation, explainability,

scalability, security, and deployment.

Appendix A: Key Implementation Details

The final implementation discussed during development includes the following important components:

- `accounts/views.py`: registration, login, profile, user management, and password-change APIs.
- `accounts/serializers.py`: `RegisterSerializer`, `ProfileSerializer`, and `UserSerializer`.
- `accounts/permissions.py`: `IsAdminUserRole` for staff-only access.
- `accounts/urls.py`: authentication, profile, user, and password routes.
- `detection/ml_model.py`: loading the saved model/vectorizer and producing prediction and decision score.
- `PredictionHistory`: persistence of news, prediction, score, and timestamp.
- `Home.jsx`: dashboard composition and derived prediction statistics.
- `NewsForm.jsx`: news submission, loading state, prediction result, and history refresh.
- `PredictionCard.jsx`: result visualization and confidence-style progress bar.
- `PieChart.jsx`: Real/Fake distribution visualization.
- `LineChart.jsx`: prediction activity over time.
- `History.jsx`: search, filter, deletion, clear-history, PDF export, and CSV export.
- `AdminUsers.jsx`: administrator user listing, search, sorting, and deletion.

Appendix B: Development Notes

- The backend project path used during development was `D:\fake_news_detection\backend`.
- A Python virtual environment was used for backend dependencies.
- Postman was used to verify API behavior.
- The frontend uses a centralized API service for backend requests.
- The project explicitly uses Django for the backend rather than Flask.

End of Project Report